FITZONE GYM MEMBERSHIP & ATTENDANCE TRACKING SYSTEM

Database Management Systems — SQL & NoSQL

Student	Ashutosh Mishra
Programme	B.Tech Computer Science & Engineering
Semester	III
Institute	ITM Skills University
Database	PostgreSQL
Database Name	fitzone_db

Project focus: database design, normalized relational schema, SQL DDL/DML, queries, constraints and trigger-based membership validation.

1. Case Study Requirements

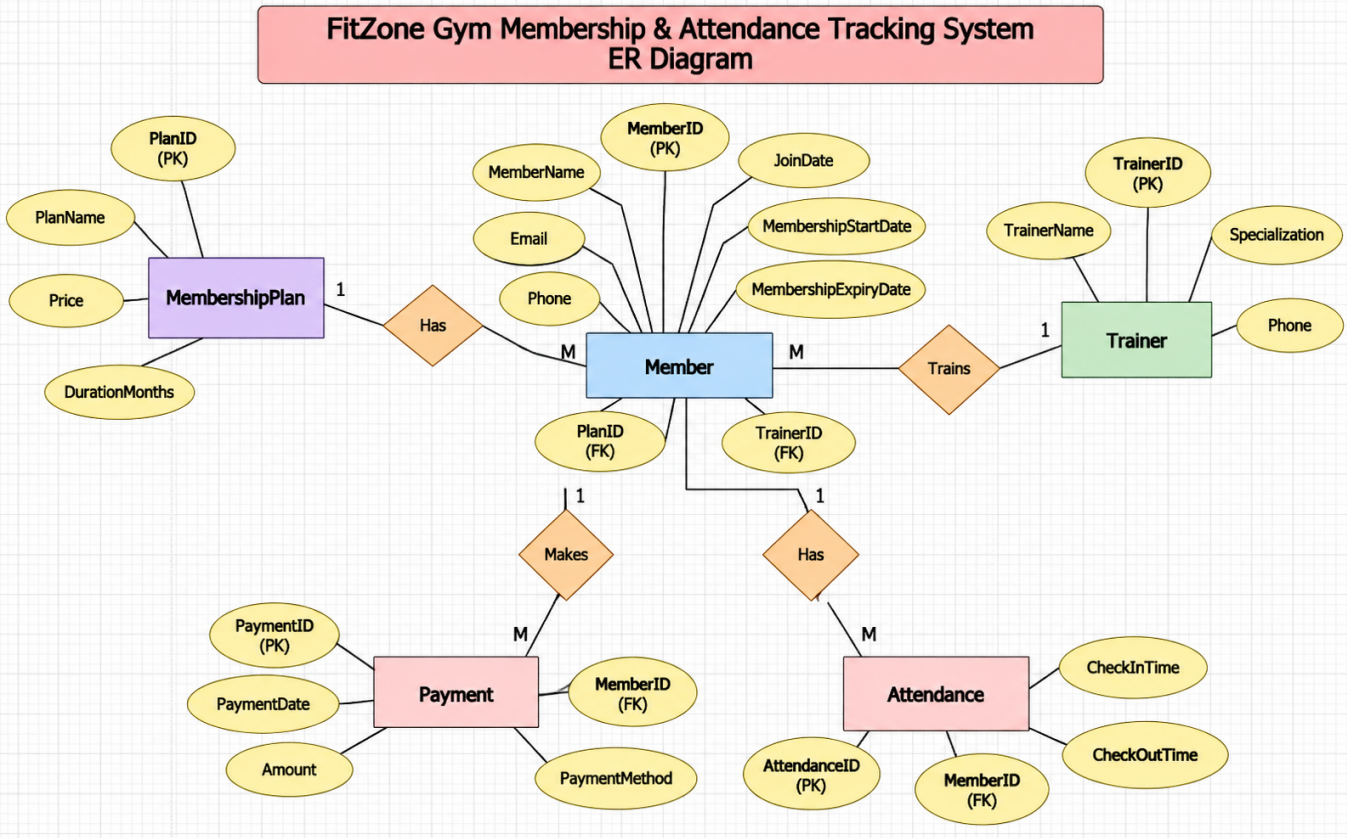
FitZone requires a database to track membership plans, members, attendance, trainers and payments, while preventing an expired member from checking in.

Required deliverables

- ER Diagram showing entities, attributes, relationships and cardinalities.
- Schema Design and Normalization with relational tables, keys and constraints.
- SQL Scripts containing DDL for table creation and DML for sample data.
- Query Set using JOINs, subqueries and aggregate functions.
- Constraint enforcement for blocking attendance/check-in for expired membership.
- Working database with sample data and documented query outputs.

2. Entity Relationship (ER) Diagram

The ER diagram models MembershipPlan, Trainer, Member, Payment and Attendance and their relationships.



Final ER diagram for the FitZone database.

3. Relational Schema

The ER model is converted into separate relational tables using primary and foreign keys.

Table	Primary Key	Foreign Key(s)	Main attributes
MembershipPlan	PlanID	—	PlanName, Price, DurationMonths
Trainer	TrainerID	—	TrainerName, Specialization, Phone
Member	MemberID	PlanID, TrainerID	MemberName, Email, Phone, JoinDate, MembershipStartDate, MembershipExp
Payment	PaymentID	MemberID	PaymentDate, Amount, PaymentMethod
Attendance	AttendanceID	MemberID	CheckInTime, CheckOutTime

- MembershipPlan → Member: one plan can be assigned to many members.
- Trainer → Member: one trainer can be associated with multiple members.
- Member → Payment: one member can have multiple payment records.
- Member → Attendance: one member can have multiple attendance records.

4. Database Creation and Table Definition

Shows creation of fitzone_db and the core tables with primary keys, NOT NULL, UNIQUE, CHECK and foreign-key constraints.

```
Last login: Mon Sep 28 10:15:24 on ttys032
ashutoshmishra@ashutoshs-MacBook-Air-2 ~ % psql postgres
psql (18.4 (Homebrew))
Type "help" for help.

postgres=# CREATE DATABASE fitzone_db;
CREATE DATABASE
postgres=# \c fitzone_db
You are now connected to database "fitzone_db" as user "ashutoshmishra".
fitzone_db=# CREATE TABLE MembershipPlan (
    PlanID SERIAL PRIMARY KEY,
    PlanName VARCHAR(50) NOT NULL UNIQUE,
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    DurationMonths INT NOT NULL CHECK (DurationMonths > 0)
);
CREATE TABLE
fitzone_db=# CREATE TABLE Trainer (
    TrainerID SERIAL PRIMARY KEY,
    TrainerName VARCHAR(100) NOT NULL,
    Specialization VARCHAR(100),
    Phone VARCHAR(15) UNIQUE
);
CREATE TABLE
fitzone_db=# CREATE TABLE Member (
    MemberID SERIAL PRIMARY KEY,
    MemberName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(15) UNIQUE,
    JoinDate DATE NOT NULL DEFAULT CURRENT_DATE,
    PlanID INT NOT NULL,
    TrainerID INT,

    CONSTRAINT fk_member_plan
        FOREIGN KEY (PlanID)
        REFERENCES MembershipPlan(PlanID),

    CONSTRAINT fk_member_trainer
        FOREIGN KEY (TrainerID)
        REFERENCES Trainer(TrainerID)
);
CREATE TABLE
fitzone_db=# CREATE TABLE Payment (
    PaymentID SERIAL PRIMARY KEY,
    MemberID INT NOT NULL,
    PaymentDate DATE NOT NULL DEFAULT CURRENT_DATE,
    Amount DECIMAL(10,2) NOT NULL CHECK (Amount > 0),
    PaymentMethod VARCHAR(20) NOT NULL
        CHECK (PaymentMethod IN ('Cash', 'Card', 'UPI', 'Bank Transfer')),

    CONSTRAINT fk_payment_member
        FOREIGN KEY (MemberID)
        REFERENCES Member(MemberID)
);
CREATE TABLE
fitzone_db=# CREATE TABLE Attendance (
    AttendanceID SERIAL PRIMARY KEY,
    MemberID INT NOT NULL,
    CheckInTime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CheckOutTime TIMESTAMP,

    CONSTRAINT fk_attendance_member
        FOREIGN KEY (MemberID)
        REFERENCES Member(MemberID),
```

Terminal evidence from the completed FitZone implementation.

5. Sample Data — Membership Plans and Trainers

Shows insertion and verification of membership plans and trainer records.

```
    CONSTRAINT fk_attendance_member
        FOREIGN KEY (MemberID)
        REFERENCES Member(MemberID),

    CONSTRAINT chk_checkout
        CHECK (
            CheckOutTime IS NULL
            OR CheckOutTime >= CheckInTime
        )
);
CREATE TABLE
fitzone_db=# ALTER TABLE Member
ADD COLUMN MembershipStartDate DATE NOT NULL DEFAULT CURRENT_DATE;

ALTER TABLE Member
ADD COLUMN MembershipExpiryDate DATE;
ALTER TABLE
ALTER TABLE
fitzone_db=# UPDATE Member m
SET MembershipExpiryDate =
    m.MembershipStartDate
    + (mp.DurationMonths * INTERVAL '1 month')
FROM MembershipPlan mp
WHERE m.PlanID = mp.PlanID;
UPDATE 0
fitzone_db=# INSERT INTO MembershipPlan
(PlanName, Price, DurationMonths)
VALUES
('Monthly', 1500, 1),
('Quarterly', 4000, 3),
('Half-Yearly', 7000, 6),
('Yearly', 12000, 12);
INSERT 0 4
fitzone_db=# SELECT * FROM MembershipPlan;
 planid |  planname   |  price   | durationmonths
--------+-------------+----------+----------------
      1 | Monthly     |  1500.00 |              1
      2 | Quarterly   |  4000.00 |              3
      3 | Half-Yearly |  7000.00 |              6
      4 | Yearly      | 12000.00 |             12
(4 rows)

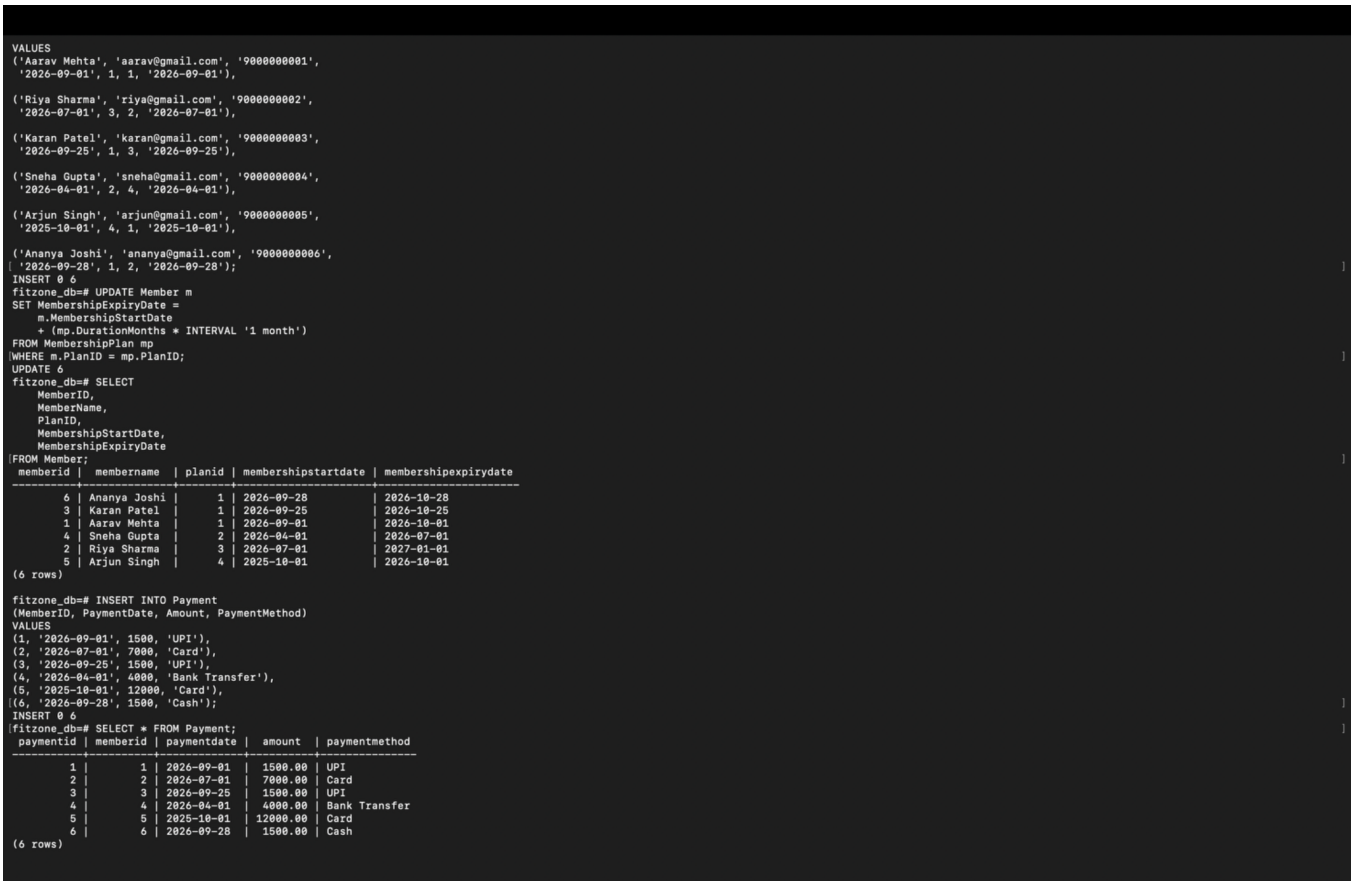
fitzone_db=# INSERT INTO Trainer
(TrainerName, Specialization, Phone)
VALUES
('Rahul Sharma', 'Strength Training', '9876543210'),
('Priya Singh', 'Yoga & Flexibility', '9876543211'),
('Aman Verma', 'Cardio Training', '9876543212'),
('Neha Patel', 'Weight Loss', '9876543213');
INSERT 0 4
fitzone_db=# SELECT * FROM Trainer;
 trainerid | trainername  |   specialization   |   phone
-----------+--------------+--------------------+------------
         1 | Rahul Sharma | Strength Training  | 9876543210
         2 | Priya Singh  | Yoga & Flexibility | 9876543211
         3 | Aman Verma   | Cardio Training    | 9876543212
         4 | Neha Patel   | Weight Loss        | 9876543213
(4 rows)

fitzone_db=# INSERT INTO Member
(MemberName, Email, Phone, JoinDate, PlanID, TrainerID, MembershipStartDate)
VALUES
('Aarav Mehta', 'aarav@gmail.com', '9000000001',
```

Terminal evidence from the completed FitZone implementation.

6. Sample Data — Members and Payments

Shows member insertion, membership-expiry calculation and payment records.



```
VALUES
('Aarav Mehta', 'aarav@gmail.com', '9000000001',
 '2026-09-01', 1, 1, '2026-09-01'),

('Riya Sharma', 'riya@gmail.com', '9000000002',
 '2026-07-01', 3, 2, '2026-07-01'),

('Karan Patel', 'karan@gmail.com', '9000000003',
 '2026-09-25', 1, 3, '2026-09-25'),

('Sneha Gupta', 'sneha@gmail.com', '9000000004',
 '2026-04-01', 2, 4, '2026-04-01'),

('Arjun Singh', 'arjun@gmail.com', '9000000005',
 '2025-10-01', 4, 1, '2025-10-01'),

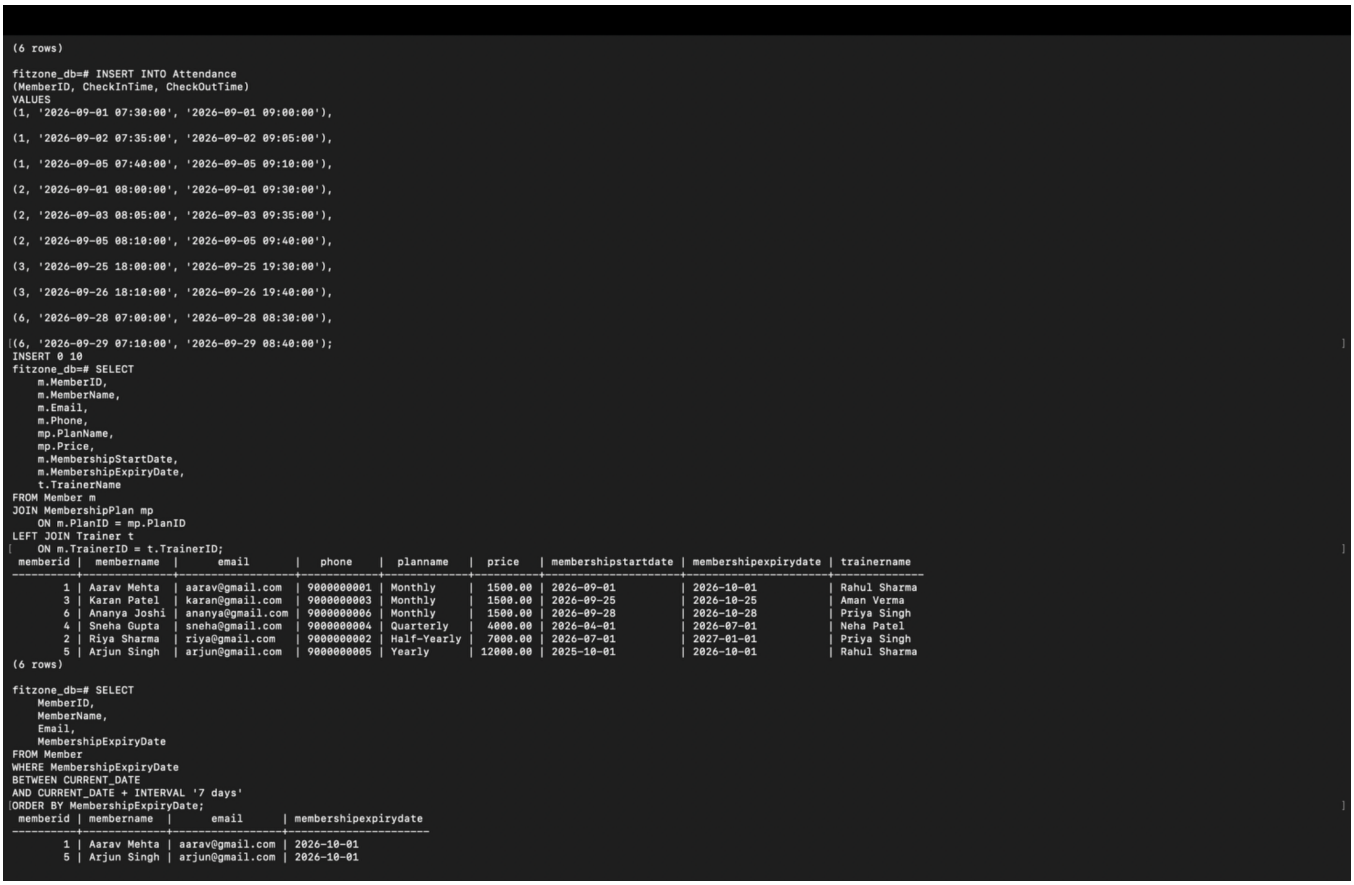
('Ananya Joshi', 'ananya@gmail.com', '9000000006',
 '2026-09-28', 1, 2, '2026-09-28');
INSERT 0 6
fitzone_db=# UPDATE Member m
SET MembershipExpiryDate =
    m.MembershipStartDate
    + (mp.DurationMonths * INTERVAL '1 month')
FROM MembershipPlan mp
WHERE m.PlanID = mp.PlanID;
UPDATE 6
fitzone_db=# SELECT
    MemberID,
    MemberName,
    PlanID,
    MembershipStartDate,
    MembershipExpiryDate
FROM Member;
 memberid |  membername  | planid | membershipstartdate | membershipexpirydate
----------+--------------+--------+---------------------+----------------------
        6 | Ananya Joshi |      1 | 2026-09-28          | 2026-10-28
        3 | Karan Patel  |      1 | 2026-09-25          | 2026-10-25
        1 | Aarav Mehta  |      1 | 2026-09-01          | 2026-10-01
        4 | Sneha Gupta  |      2 | 2026-04-01          | 2026-07-01
        2 | Riya Sharma  |      3 | 2026-07-01          | 2027-01-01
        5 | Arjun Singh  |      4 | 2025-10-01          | 2026-10-01
(6 rows)

fitzone_db=# INSERT INTO Payment
(MemberID, PaymentDate, Amount, PaymentMethod)
VALUES
(1, '2026-09-01', 1500, 'UPI'),
(2, '2026-07-01', 7000, 'Card'),
(3, '2026-09-25', 1500, 'UPI'),
(4, '2026-04-01', 4000, 'Bank Transfer'),
(5, '2025-10-01', 12000, 'Card'),
(6, '2026-09-28', 1500, 'Cash');
INSERT 0 6
fitzone_db=# SELECT * FROM Payment;
 paymentid | memberid | paymentdate |  amount  | paymentmethod
-----------+----------+-------------+----------+---------------
         1 |        1 | 2026-09-01  |  1500.00 | UPI
         2 |        2 | 2026-07-01  |  7000.00 | Card
         3 |        3 | 2026-09-25  |  1500.00 | UPI
         4 |        4 | 2026-04-01  |  4000.00 | Bank Transfer
         5 |        5 | 2025-10-01  | 12000.00 | Card
         6 |        6 | 2026-09-28  |  1500.00 | Cash
(6 rows)
```

Terminal evidence from the completed FitZone implementation.

7. Attendance Data and Joined Output

Shows attendance insertion and a JOIN combining member, plan and trainer information.



```
(6 rows)

fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(1, '2026-09-01 07:30:00', '2026-09-01 09:00:00'),

(1, '2026-09-02 07:35:00', '2026-09-02 09:05:00'),

(1, '2026-09-05 07:40:00', '2026-09-05 09:10:00'),

(2, '2026-09-01 08:00:00', '2026-09-01 09:30:00'),

(2, '2026-09-03 08:05:00', '2026-09-03 09:35:00'),

(2, '2026-09-05 08:10:00', '2026-09-05 09:40:00'),

(3, '2026-09-25 18:00:00', '2026-09-25 19:30:00'),

(3, '2026-09-26 18:10:00', '2026-09-26 19:40:00'),

(6, '2026-09-28 07:00:00', '2026-09-28 08:30:00'),

(6, '2026-09-29 07:10:00', '2026-09-29 08:40:00');
INSERT 0 10
fitzone_db=# SELECT
    m.MemberID,
    m.MemberName,
    m.Email,
    m.Phone,
    mp.PlanName,
    mp.Price,
    m.MembershipStartDate,
    m.MembershipExpiryDate,
    t.TrainerName
FROM Member m
JOIN MembershipPlan mp
    ON m.PlanID = mp.PlanID
LEFT JOIN Trainer t
    ON m.TrainerID = t.TrainerID;
 memberid |  membername  |      email       |   phone    |  planname   |  price   | membershipstartdate | membershipexpirydate | trainername
----------+--------------+------------------+------------+-------------+----------+---------------------+----------------------+--------------
        1 | Aarav Mehta  | aarav@gmail.com  | 9000000001 | Monthly     |  1500.00 | 2026-09-01          | 2026-10-01           | Rahul Sharma
        3 | Karan Patel  | karan@gmail.com  | 9000000003 | Monthly     |  1500.00 | 2026-09-25          | 2026-10-25           | Aman Verma
        6 | Ananya Joshi | ananya@gmail.com | 9000000006 | Monthly     |  1500.00 | 2026-09-28          | 2026-10-28           | Priya Singh
        4 | Sneha Gupta  | sneha@gmail.com  | 9000000004 | Quarterly   |  4000.00 | 2026-04-01          | 2026-07-01           | Neha Patel
        2 | Riya Sharma  | riya@gmail.com   | 9000000002 | Half-Yearly |  7000.00 | 2026-07-01          | 2027-01-01           | Priya Singh
        5 | Arjun Singh  | arjun@gmail.com  | 9000000005 | Yearly      | 12000.00 | 2025-10-01          | 2026-10-01           | Rahul Sharma
(6 rows)

fitzone_db=# SELECT
    MemberID,
    MemberName,
    Email,
    MembershipExpiryDate
FROM Member
WHERE MembershipExpiryDate
BETWEEN CURRENT_DATE
AND CURRENT_DATE + INTERVAL '7 days'
ORDER BY MembershipExpiryDate;
 memberid | membername  |      email      | membershipexpirydate
----------+-------------+-----------------+----------------------
        1 | Aarav Mehta | aarav@gmail.com | 2026-10-01
        5 | Arjun Singh | arjun@gmail.com | 2026-10-01
```

Terminal evidence from the completed FitZone implementation.

8. Membership Expiry and Attendance Aggregation

Shows expiry-related queries and attendance aggregation using COUNT and GROUP BY.

```
        5 | Arjun Singh | arjun@gmail.com | 2026-10-01
(2 rows)

fitzone_db=# SELECT
    MemberID,
    MemberName,
    MembershipExpiryDate
FROM Member
WHERE MembershipExpiryDate < CURRENT_DATE;
 memberid | membername  | membershipexpirydate
----------+-------------+----------------------
        4 | Sneha Gupta | 2026-07-01
(1 row)

fitzone_db=# SELECT
    MemberID,
    MemberName,
    MembershipExpiryDate
FROM Member
WHERE MembershipExpiryDate >= CURRENT_DATE;
 memberid |  membername  | membershipexpirydate
----------+--------------+----------------------
        6 | Ananya Joshi | 2026-10-28
        3 | Karan Patel  | 2026-10-25
        1 | Aarav Mehta  | 2026-10-01
        2 | Riya Sharma  | 2027-01-01
        5 | Arjun Singh  | 2026-10-01
(5 rows)

fitzone_db=# SELECT
    m.MemberID,
    m.MemberName,
    DATE_TRUNC('month', a.CheckInTime) AS AttendanceMonth,
    COUNT(*) AS AttendanceCount
FROM Attendance a
JOIN Member m
    ON a.MemberID = m.MemberID
GROUP BY
    m.MemberID,
    m.MemberName,
    DATE_TRUNC('month', a.CheckInTime)
ORDER BY
    AttendanceMonth,
    AttendanceCount DESC;
 memberid |  membername  |   attendancemonth   | attendancecount
----------+--------------+---------------------+-----------------
        2 | Riya Sharma  | 2026-09-01 00:00:00 |               3
        1 | Aarav Mehta  | 2026-09-01 00:00:00 |               3
        3 | Karan Patel  | 2026-09-01 00:00:00 |               2
        6 | Ananya Joshi | 2026-09-01 00:00:00 |               2
(4 rows)

fitzone_db=# SELECT
    m.MemberID,
    m.MemberName,
    COUNT(a.AttendanceID) AS TotalVisits
FROM Member m
LEFT JOIN Attendance a
    ON m.MemberID = a.MemberID
GROUP BY
    m.MemberID,
    m.MemberName
ORDER BY TotalVisits DESC;
 memberid |  membername  | totalvisits
```

Terminal evidence from the completed FitZone implementation.

9. Attendance, Trainer and Payment Queries

Shows total visits, detailed attendance and trainer-wise member counts.

```
    ON m.MemberID = a.MemberID
GROUP BY
    m.MemberID,
    m.MemberName
ORDER BY TotalVisits DESC;
 memberid |  membername  | totalvisits
----------+--------------+-------------
        2 | Riya Sharma  |           3
        1 | Aarav Mehta  |           3
        3 | Karan Patel  |           2
        6 | Ananya Joshi |           2
        5 | Arjun Singh  |           0
        4 | Sneha Gupta  |           0
(6 rows)

fitzone_db=# SELECT
    a.AttendanceID,
    m.MemberName,
    a.CheckInTime,
    a.CheckOutTime
FROM Attendance a
JOIN Member m
    ON a.MemberID = m.MemberID
ORDER BY a.CheckInTime;
 attendanceid |  membername  |     checkintime     |    checkouttime
--------------+--------------+---------------------+---------------------
            1 | Aarav Mehta  | 2026-09-01 07:30:00 | 2026-09-01 09:00:00
            4 | Riya Sharma  | 2026-09-01 08:00:00 | 2026-09-01 09:30:00
            2 | Aarav Mehta  | 2026-09-02 07:35:00 | 2026-09-02 09:05:00
            5 | Riya Sharma  | 2026-09-03 08:05:00 | 2026-09-03 09:35:00
            3 | Aarav Mehta  | 2026-09-05 07:40:00 | 2026-09-05 09:10:00
            6 | Riya Sharma  | 2026-09-05 08:10:00 | 2026-09-05 09:40:00
            7 | Karan Patel  | 2026-09-25 18:00:00 | 2026-09-25 19:30:00
            8 | Karan Patel  | 2026-09-26 18:10:00 | 2026-09-26 19:40:00
            9 | Ananya Joshi | 2026-09-28 07:00:00 | 2026-09-28 08:30:00
           10 | Ananya Joshi | 2026-09-29 07:10:00 | 2026-09-29 08:40:00
(10 rows)

fitzone_db=# SELECT
    t.TrainerID,
    t.TrainerName,
    COUNT(m.MemberID) AS TotalMembers
FROM Trainer t
LEFT JOIN Member m
    ON t.TrainerID = m.TrainerID
GROUP BY
    t.TrainerID,
    t.TrainerName
ORDER BY TotalMembers DESC;
 trainerid | trainername  | totalmembers
-----------+--------------+--------------
         2 | Priya Singh  |            2
         1 | Rahul Sharma |            2
         3 | Aman Verma   |            1
         4 | Neha Patel   |            1
(4 rows)

fitzone_db=# SELECT
    p.PaymentID,
    m.MemberName,
    p.PaymentDate,
    p.Amount,
    p.PaymentMethod
FROM Payment p
```

Terminal evidence from the completed FitZone implementation.

10. Payment Summary and Trigger Function

Shows total revenue, payment-method aggregation and creation of check_active_membership().

```
FROM Payment p
JOIN Member m
    ON p.MemberID = m.MemberID
ORDER BY p.PaymentDate DESC;
 paymentid |  membername  | paymentdate |  amount  | paymentmethod
-----------+--------------+-------------+----------+---------------
         6 | Ananya Joshi | 2026-09-28  |  1500.00 | Cash
         3 | Karan Patel  | 2026-09-25  |  1500.00 | UPI
         1 | Aarav Mehta  | 2026-09-01  |  1500.00 | UPI
         2 | Riya Sharma  | 2026-07-01  |  7000.00 | Card
         4 | Sneha Gupta  | 2026-04-01  |  4000.00 | Bank Transfer
         5 | Arjun Singh  | 2025-10-01  | 12000.00 | Card
(6 rows)

fitzone_db=# SELECT
    SUM(Amount) AS TotalRevenue
FROM Payment;
 totalrevenue
--------------
     27500.00
(1 row)

fitzone_db=# SELECT
    PaymentMethod,
    COUNT(*) AS NumberOfPayments,
    SUM(Amount) AS TotalAmount
FROM Payment
GROUP BY PaymentMethod
ORDER BY TotalAmount DESC;
 paymentmethod | numberofpayments | totalamount
---------------+------------------+-------------
 Card          |                2 |    19000.00
 Bank Transfer |                1 |     4000.00
 UPI           |                2 |     3000.00
 Cash          |                1 |     1500.00
(4 rows)

fitzone_db=# CREATE OR REPLACE FUNCTION check_active_membership()
RETURNS TRIGGER
AS $$
DECLARE
    expiry_date DATE;
BEGIN

    SELECT MembershipExpiryDate
    INTO expiry_date
    FROM Member
    WHERE MemberID = NEW.MemberID;

    IF expiry_date IS NULL THEN
        RAISE EXCEPTION
        'Membership expiry date not found for MemberID %',
        NEW.MemberID;
    END IF;

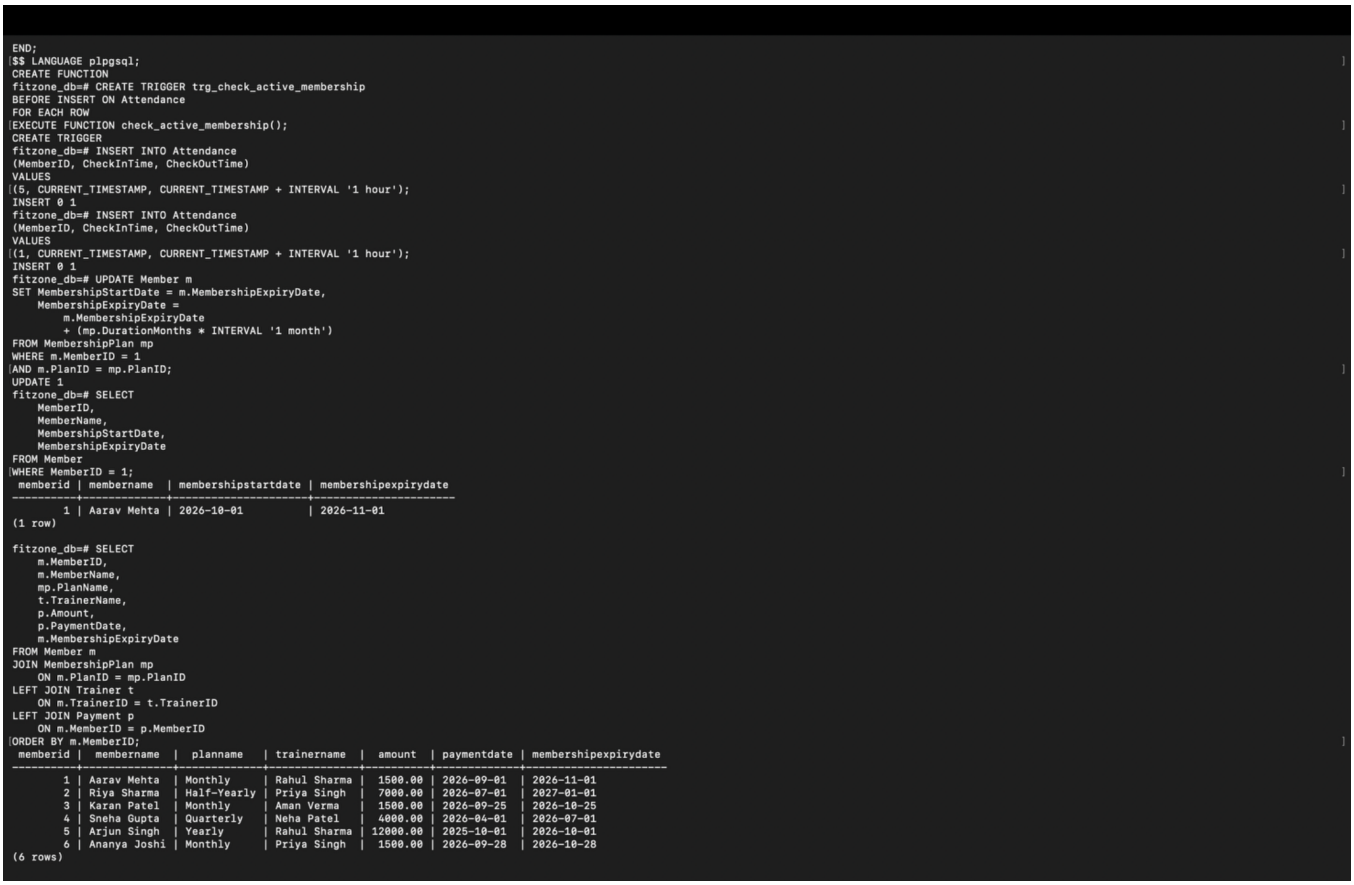
    IF expiry_date < NEW.CheckInTime::DATE THEN
        RAISE EXCEPTION
        'Attendance rejected: Membership of MemberID % expired on %',
        NEW.MemberID,
        expiry_date;
    END IF;

    RETURN NEW;
END;
```

Terminal evidence from the completed FitZone implementation.

11. Trigger Creation and Membership Update

Shows creation of trg_check_active_membership and membership update/testing.



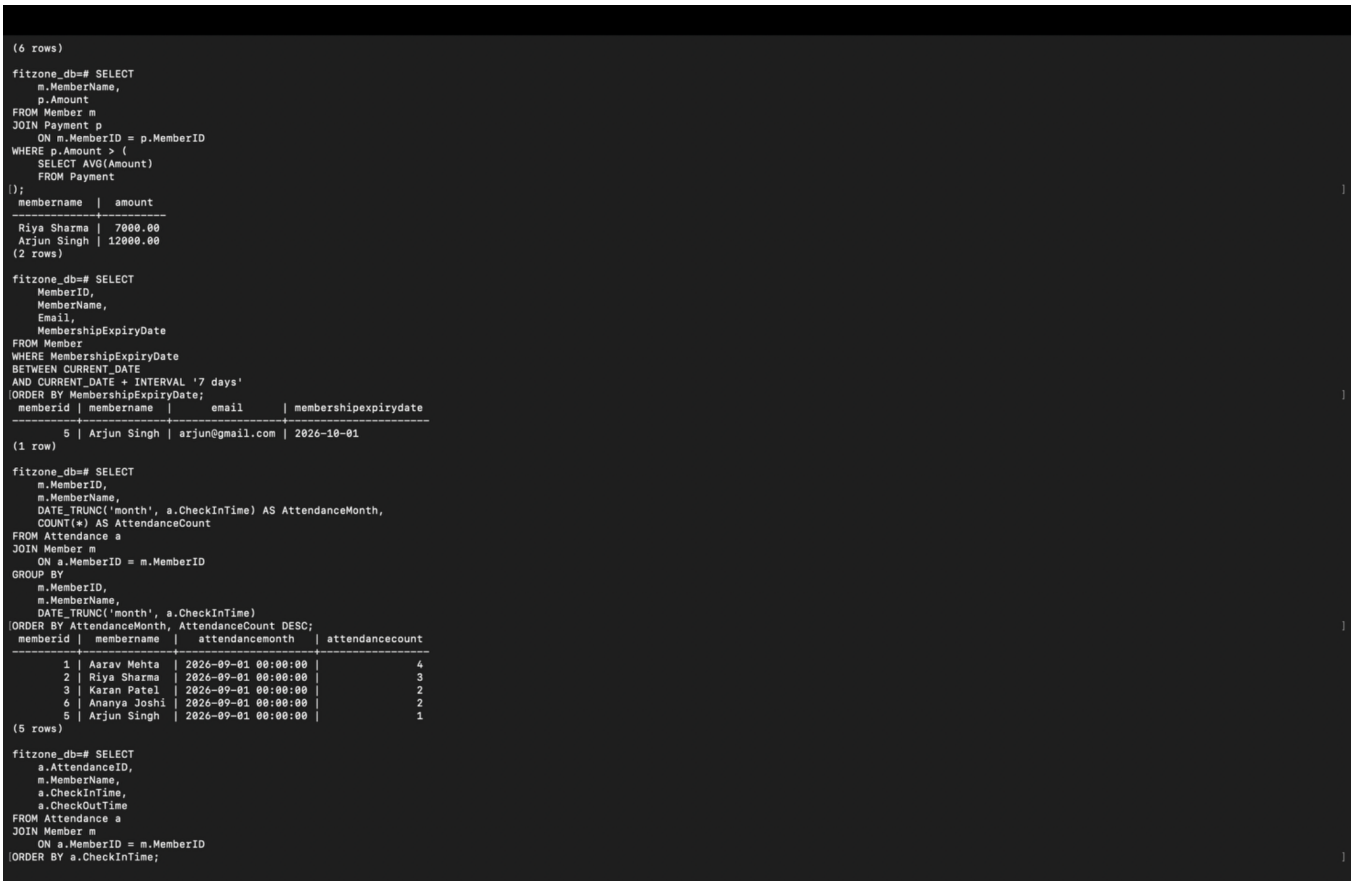
```
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
fitzone_db=# CREATE TRIGGER trg_check_active_membership
BEFORE INSERT ON Attendance
FOR EACH ROW
EXECUTE FUNCTION check_active_membership();
CREATE TRIGGER
fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(5, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP + INTERVAL '1 hour');
INSERT 0 1
fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP + INTERVAL '1 hour');
INSERT 0 1
fitzone_db=# UPDATE Member m
SET MembershipStartDate = m.MembershipExpiryDate,
    MembershipExpiryDate =
        m.MembershipExpiryDate
        + (mp.DurationMonths * INTERVAL '1 month')
FROM MembershipPlan mp
WHERE m.MemberID = 1
AND m.PlanID = mp.PlanID;
UPDATE 1
fitzone_db=# SELECT
    MemberID,
    MemberName,
    MembershipStartDate,
    MembershipExpiryDate
FROM Member
WHERE MemberID = 1;
 memberid | membername  | membershipstartdate | membershipexpirydate
----------+-------------+---------------------+----------------------
        1 | Aarav Mehta | 2026-10-01          | 2026-11-01
(1 row)

fitzone_db=# SELECT
    m.MemberID,
    m.MemberName,
    mp.PlanName,
    t.TrainerName,
    p.Amount,
    p.PaymentDate,
    m.MembershipExpiryDate
FROM Member m
JOIN MembershipPlan mp
    ON m.PlanID = mp.PlanID
LEFT JOIN Trainer t
    ON m.TrainerID = t.TrainerID
LEFT JOIN Payment p
    ON m.MemberID = p.MemberID
ORDER BY m.MemberID;
 memberid |  membername  |  planname   | trainername  |  amount  | paymentdate | membershipexpirydate
----------+--------------+-------------+--------------+----------+-------------+----------------------
        1 | Aarav Mehta  | Monthly     | Rahul Sharma |  1500.00 | 2026-09-01  | 2026-11-01
        2 | Riya Sharma  | Half-Yearly | Priya Singh  |  7000.00 | 2026-07-01  | 2027-01-01
        3 | Karan Patel  | Monthly     | Aman Verma   |  1500.00 | 2026-09-25  | 2026-10-25
        4 | Sneha Gupta  | Quarterly   | Neha Patel   |  4000.00 | 2026-04-01  | 2026-07-01
        5 | Arjun Singh  | Yearly      | Rahul Sharma | 12000.00 | 2025-10-01  | 2026-10-01
        6 | Ananya Joshi | Monthly     | Priya Singh  |  1500.00 | 2026-09-28  | 2026-10-28
(6 rows)
```

Terminal evidence from the completed FitZone implementation.

12. Joined Report and Average-Payment Subquery

Shows a combined report and a subquery using AVG to find payments above the average.



```
(6 rows)

fitzone_db=# SELECT
    m.MemberName,
    p.Amount
FROM Member m
JOIN Payment p
    ON m.MemberID = p.MemberID
WHERE p.Amount > (
    SELECT AVG(Amount)
    FROM Payment
);
 membername  |  amount
-------------+----------
 Riya Sharma |  7000.00
 Arjun Singh | 12000.00
(2 rows)

fitzone_db=# SELECT
    MemberID,
    MemberName,
    Email,
    MembershipExpiryDate
FROM Member
WHERE MembershipExpiryDate
BETWEEN CURRENT_DATE
AND CURRENT_DATE + INTERVAL '7 days'
ORDER BY MembershipExpiryDate;
 memberid | membername  |      email      | membershipexpirydate
----------+-------------+-----------------+----------------------
        5 | Arjun Singh | arjun@gmail.com | 2026-10-01
(1 row)

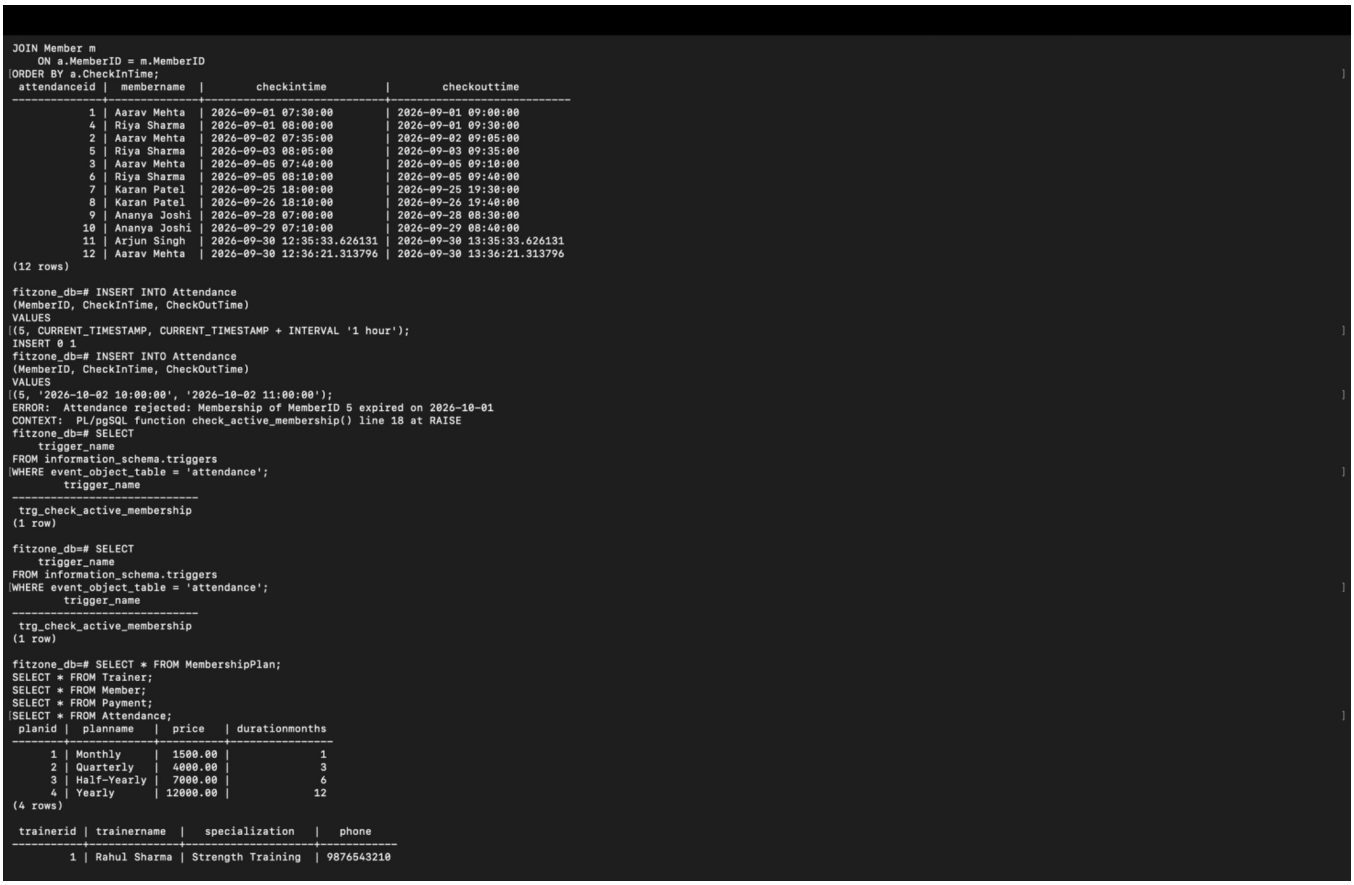
fitzone_db=# SELECT
    m.MemberID,
    m.MemberName,
    DATE_TRUNC('month', a.CheckInTime) AS AttendanceMonth,
    COUNT(*) AS AttendanceCount
FROM Attendance a
JOIN Member m
    ON a.MemberID = m.MemberID
GROUP BY
    m.MemberID,
    m.MemberName,
    DATE_TRUNC('month', a.CheckInTime)
ORDER BY AttendanceMonth, AttendanceCount DESC;
 memberid |  membername  |    attendancemonth    | attendancecount
----------+--------------+-----------------------+-----------------
        1 | Aarav Mehta  | 2026-09-01 00:00:00 |               4
        2 | Riya Sharma  | 2026-09-01 00:00:00 |               3
        3 | Karan Patel  | 2026-09-01 00:00:00 |               2
        6 | Ananya Joshi | 2026-09-01 00:00:00 |               2
        5 | Arjun Singh  | 2026-09-01 00:00:00 |               1
(5 rows)

fitzone_db=# SELECT
    a.AttendanceID,
    m.MemberName,
    a.CheckInTime,
    a.CheckOutTime
FROM Attendance a
JOIN Member m
    ON a.MemberID = m.MemberID
ORDER BY a.CheckInTime;
```

Terminal evidence from the completed FitZone implementation.

13. Additional Query Outputs

Shows further membership-expiry and attendance query results.



```
JOIN Member m
    ON a.MemberID = m.MemberID
ORDER BY a.CheckInTime;
 attendanceid |  membername  |        checkintime         |        checkouttime
--------------+--------------+----------------------------+----------------------------
            1 | Aarav Mehta  | 2026-09-01 07:30:00        | 2026-09-01 09:00:00
            4 | Riya Sharma  | 2026-09-01 08:00:00        | 2026-09-01 09:30:00
            2 | Aarav Mehta  | 2026-09-02 07:35:00        | 2026-09-02 09:05:00
            5 | Riya Sharma  | 2026-09-03 08:05:00        | 2026-09-03 09:35:00
            3 | Aarav Mehta  | 2026-09-05 07:40:00        | 2026-09-05 09:10:00
            6 | Riya Sharma  | 2026-09-05 08:10:00        | 2026-09-05 09:40:00
            7 | Karan Patel  | 2026-09-25 18:00:00        | 2026-09-25 19:30:00
            8 | Karan Patel  | 2026-09-26 18:10:00        | 2026-09-26 19:40:00
            9 | Ananya Joshi | 2026-09-28 07:00:00        | 2026-09-28 08:30:00
           10 | Ananya Joshi | 2026-09-29 07:10:00        | 2026-09-29 08:40:00
           11 | Arjun Singh  | 2026-09-30 12:35:33.626131 | 2026-09-30 13:35:33.626131
           12 | Aarav Mehta  | 2026-09-30 12:36:21.313796 | 2026-09-30 13:36:21.313796
(12 rows)

fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(5, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP + INTERVAL '1 hour');
INSERT 0 1
fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(5, '2026-10-02 10:00:00', '2026-10-02 11:00:00');
ERROR:  Attendance rejected: Membership of MemberID 5 expired on 2026-10-01
CONTEXT:  PL/pgSQL function check_active_membership() line 18 at RAISE
fitzone_db=# SELECT
    trigger_name
FROM information_schema.triggers
WHERE event_object_table = 'attendance';
        trigger_name
-----------------------------
 trg_check_active_membership
(1 row)

fitzone_db=# SELECT
    trigger_name
FROM information_schema.triggers
WHERE event_object_table = 'attendance';
        trigger_name
-----------------------------
 trg_check_active_membership
(1 row)

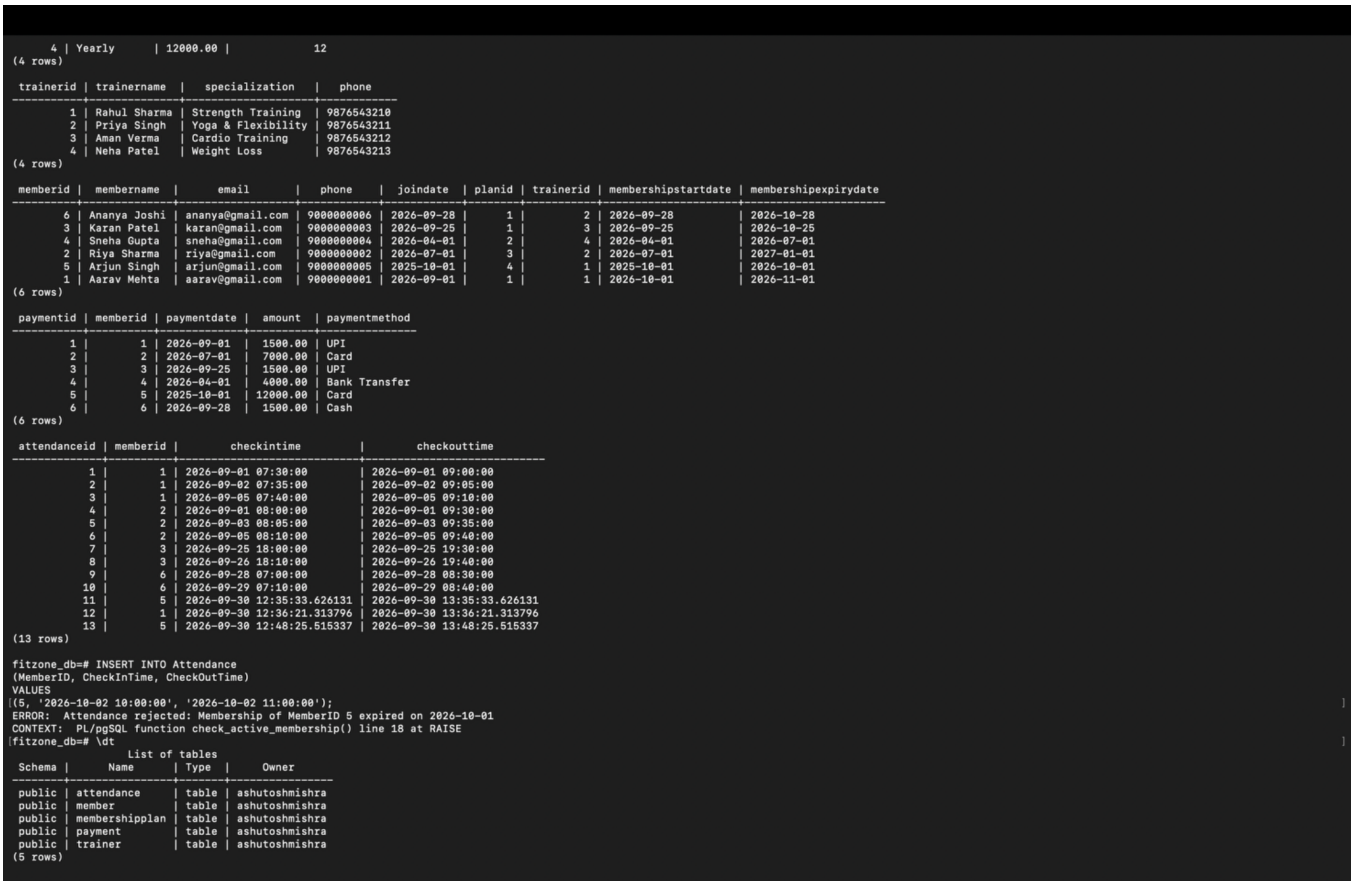
fitzone_db=# SELECT * FROM MembershipPlan;
SELECT * FROM Trainer;
SELECT * FROM Member;
SELECT * FROM Payment;
SELECT * FROM Attendance;
 planid |  planname   |  price   | durationmonths
--------+-------------+----------+----------------
      1 | Monthly     |  1500.00 |              1
      2 | Quarterly   |  4000.00 |              3
      3 | Half-Yearly |  7000.00 |              6
      4 | Yearly      | 12000.00 |             12
(4 rows)

 trainerid | trainername |   specialization   |   phone
-----------+-------------+--------------------+------------
         1 | Rahul Sharma | Strength Training  | 9876543210
```

Terminal evidence from the completed FitZone implementation.

14. Trigger Test and Final Database State

Shows the expired-membership attendance rejection and final list of five database tables.



```
      4 | Yearly      | 12000.00 |            12
(4 rows)

 trainerid | trainername  |   specialization   |   phone
-----------+--------------+--------------------+------------
         1 | Rahul Sharma | Strength Training  | 9876543210
         2 | Priya Singh  | Yoga & Flexibility | 9876543211
         3 | Aman Verma   | Cardio Training    | 9876543212
         4 | Neha Patel   | Weight Loss        | 9876543213
(4 rows)

 memberid |  membername  |      email       |   phone    |  joindate  | planid | trainerid | membershipstartdate | membershipexpirydate
----------+--------------+------------------+------------+------------+--------+-----------+---------------------+----------------------
        6 | Ananya Joshi | ananya@gmail.com | 9000000006 | 2026-09-28 |      1 |         2 | 2026-09-28          | 2026-10-28
        3 | Karan Patel  | karan@gmail.com  | 9000000003 | 2026-09-25 |      1 |         3 | 2026-09-25          | 2026-10-25
        4 | Sneha Gupta  | sneha@gmail.com  | 9000000004 | 2026-04-01 |      2 |         4 | 2026-04-01          | 2026-07-01
        2 | Riya Sharma  | riya@gmail.com   | 9000000002 | 2026-07-01 |      3 |         2 | 2026-07-01          | 2027-01-01
        5 | Arjun Singh  | arjun@gmail.com  | 9000000005 | 2025-10-01 |      4 |         1 | 2025-10-01          | 2026-10-01
        1 | Aarav Mehta  | aarav@gmail.com  | 9000000001 | 2026-09-01 |      1 |         1 | 2026-10-01          | 2026-11-01
(6 rows)

 paymentid | memberid | paymentdate |  amount  | paymentmethod
-----------+----------+-------------+----------+---------------
         1 |        1 | 2026-09-01  |  1500.00 | UPI
         2 |        2 | 2026-07-01  |  7000.00 | Card
         3 |        3 | 2026-09-25  |  1500.00 | UPI
         4 |        4 | 2026-04-01  |  4000.00 | Bank Transfer
         5 |        5 | 2025-10-01  | 12000.00 | Card
         6 |        6 | 2026-09-28  |  1500.00 | Cash
(6 rows)

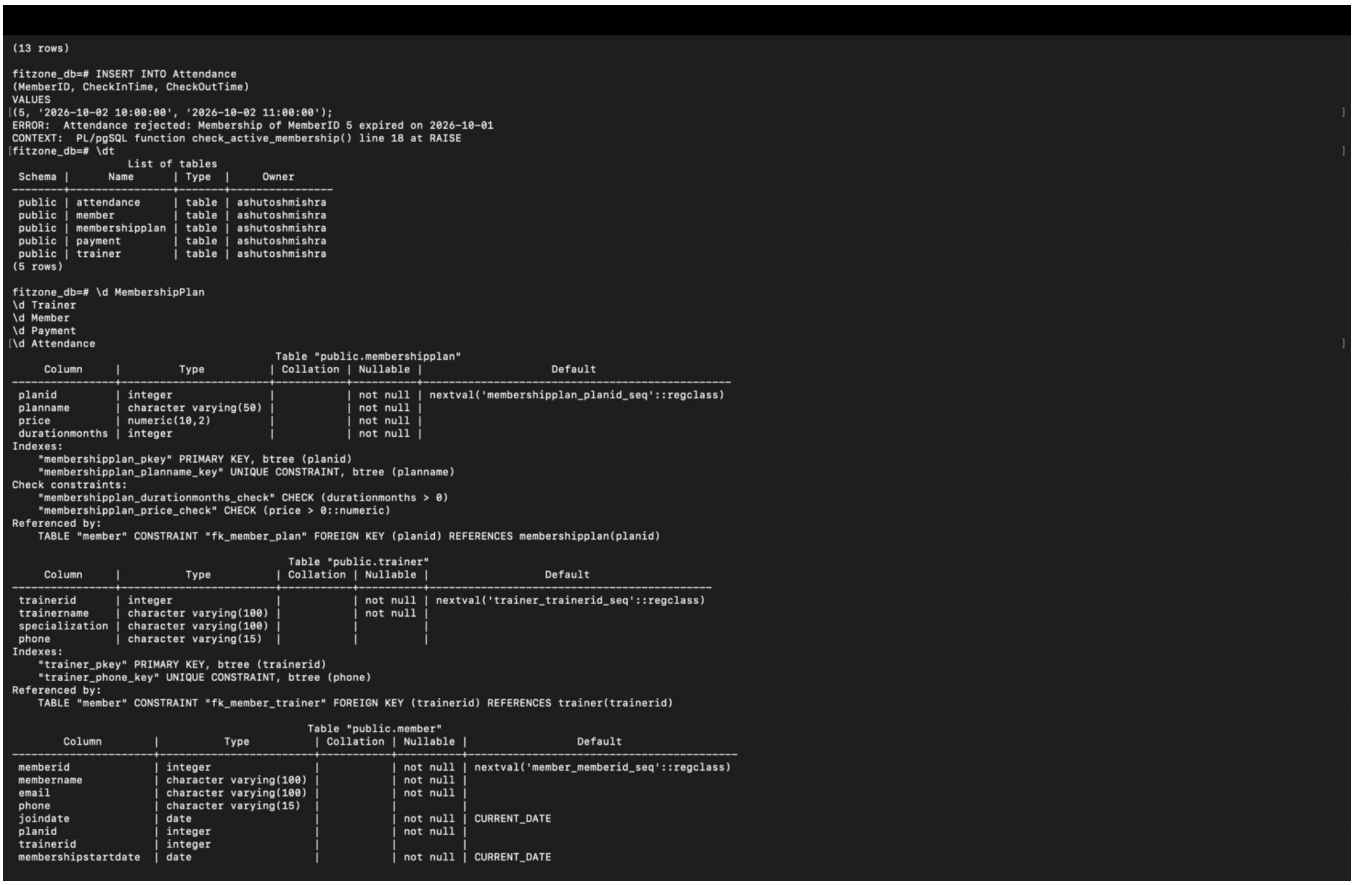
 attendanceid | memberid |        checkintime         |        checkouttime
--------------+----------+----------------------------+----------------------------
            1 |        1 | 2026-09-01 07:30:00        | 2026-09-01 09:00:00
            2 |        1 | 2026-09-02 07:35:00        | 2026-09-02 09:05:00
            3 |        1 | 2026-09-05 07:40:00        | 2026-09-05 09:10:00
            4 |        2 | 2026-09-01 08:00:00        | 2026-09-01 09:30:00
            5 |        2 | 2026-09-03 08:05:00        | 2026-09-03 09:35:00
            6 |        2 | 2026-09-05 08:10:00        | 2026-09-05 09:40:00
            7 |        3 | 2026-09-25 18:00:00        | 2026-09-25 19:30:00
            8 |        3 | 2026-09-26 18:10:00        | 2026-09-26 19:40:00
            9 |        6 | 2026-09-28 07:00:00        | 2026-09-28 08:30:00
           10 |        6 | 2026-09-29 07:10:00        | 2026-09-29 08:40:00
           11 |        5 | 2026-09-30 12:35:33.626131 | 2026-09-30 13:35:33.626131
           12 |        1 | 2026-09-30 12:36:21.313796 | 2026-09-30 13:36:21.313796
           13 |        5 | 2026-09-30 12:48:25.515337 | 2026-09-30 13:48:25.515337
(13 rows)

fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(5, '2026-10-02 10:00:00', '2026-10-02 11:00:00');
ERROR:  Attendance rejected: Membership of MemberID 5 expired on 2026-10-01
CONTEXT:  PL/pgSQL function check_active_membership() line 18 at RAISE
fitzone_db=# \dt
                List of tables
 Schema |      Name      | Type  |     Owner
--------+----------------+-------+----------------
 public | attendance     | table | ashutoshmishra
 public | member         | table | ashutoshmishra
 public | membershipplan | table | ashutoshmishra
 public | payment        | table | ashutoshmishra
 public | trainer        | table | ashutoshmishra
(5 rows)
```

Terminal evidence from the completed FitZone implementation.

15. Main Schema Verification — \d Table Definitions

This main verification screenshot confirms the final structure of MembershipPlan, Trainer, Member, Payment and Attendance. It shows columns, data types, primary keys, UNIQUE constraints, CHECK constraints, foreign keys and the Attendance trigger.



```
(13 rows)

fitzone_db=# INSERT INTO Attendance
(MemberID, CheckInTime, CheckOutTime)
VALUES
(5, '2026-10-02 10:00:00', '2026-10-02 11:00:00');
ERROR:  Attendance rejected: Membership of MemberID 5 expired on 2026-10-01
CONTEXT:  PL/pgSQL function check_active_membership() line 18 at RAISE
fitzone_db=# \dt
              List of tables
 Schema |       Name       | Type  |     Owner
--------+------------------+-------+-----------------
 public | attendance       | table | ashutoshmishra
 public | member           | table | ashutoshmishra
 public | membershipplan   | table | ashutoshmishra
 public | payment          | table | ashutoshmishra
 public | trainer          | table | ashutoshmishra
(5 rows)

fitzone_db=# \d MembershipPlan
\d Trainer
\d Member
\d Payment
\d Attendance
                                      Table "public.membershipplan"
     Column      |         Type          | Collation | Nullable |                    Default
-----------------+-----------------------+-----------+----------+-----------------------------------------------
 planid          | integer               |           | not null | nextval('membershipplan_planid_seq'::regclass)
 planname        | character varying(50) |           | not null |
 price           | numeric(10,2)         |           | not null |
 durationmonths | integer               |           | not null |
Indexes:
    "membershipplan_pkey" PRIMARY KEY, btree (planid)
    "membershipplan_planname_key" UNIQUE CONSTRAINT, btree (planname)
Check constraints:
    "membershipplan_durationmonths_check" CHECK (durationmonths > 0)
    "membershipplan_price_check" CHECK (price > 0::numeric)
Referenced by:
    TABLE "member" CONSTRAINT "fk_member_plan" FOREIGN KEY (planid) REFERENCES membershipplan(planid)

                                       Table "public.trainer"
     Column     |          Type          | Collation | Nullable |                  Default
----------------+------------------------+-----------+----------+-------------------------------------------
 trainerid      | integer                |           | not null | nextval('trainer_trainerid_seq'::regclass)
 trainername    | character varying(100) |           | not null |
 specialization | character varying(100) |           |          |
 phone          | character varying(15)  |           |          |
Indexes:
    "trainer_pkey" PRIMARY KEY, btree (trainerid)
    "trainer_phone_key" UNIQUE CONSTRAINT, btree (phone)
Referenced by:
    TABLE "member" CONSTRAINT "fk_member_trainer" FOREIGN KEY (trainerid) REFERENCES trainer(trainerid)

                                       Table "public.member"
       Column        |          Type          | Collation | Nullable |                  Default
---------------------+------------------------+-----------+----------+-------------------------------------------
 memberid            | integer                |           | not null | nextval('member_memberid_seq'::regclass)
 membername          | character varying(100) |           | not null |
 email               | character varying(100) |           | not null |
 phone               | character varying(15)  |           |          |
 joindate            | date                   |           | not null | CURRENT_DATE
 planid              | integer                |           | not null |
 trainerid           | integer                |           |          |
 membershipstartdate | date                   |           | not null | CURRENT_DATE
```

Main terminal screenshot: complete PostgreSQL table definitions and constraints.

16. Main Trigger Verification — information_schema.triggers

This screenshot directly verifies that trg_check_active_membership exists on the Attendance table as a BEFORE INSERT trigger and executes the check_active_membership() function.



Main terminal screenshot: trigger metadata verification.

17. Constraint & Trigger Enforcement

Primary keys uniquely identify records. Foreign keys maintain relationships. UNIQUE constraints prevent duplicates, while CHECK constraints validate values such as positive payment amounts and valid checkout times.

The trigger trg_check_active_membership executes BEFORE an Attendance INSERT. It checks MembershipExpiryDate and rejects a check-in when the membership has expired.

Verified test: MemberID 5 attempted attendance on 2026-10-02 while the membership expired on 2026-10-01. PostgreSQL rejected the insertion with the trigger error.

18. Conclusion

The FitZone database was implemented in PostgreSQL with the required ER model, relational schema, DDL/DML, JOIN queries, aggregate queries, subquery analysis, integrity constraints and trigger-based expired-membership enforcement.

All required case-study deliverables are represented in the report with terminal evidence and explanations in sequence.